LLM-Assisted Annotation of Singaporean Code-Mixed Chat

Nura Tamton

This report covers the dataset, the two annotation schemes, the pipeline and its results, and next steps for the LLM assisted annotation pipeline task for code mixed chat.

1 The Dataset

The corpus comprises 50 conversations and 6,483 non-empty messages of real Singaporean casual chat, mixing English with Singlish, Malay, and Chinese. The pipeline consumes two columns per conversation, `final_message_list` (the ordered messages) and `final_speaker_tracking` (the speaker per turn), together with relationship metadata attached at the conversation level. The observations below are computed from the corpus and motivate the scheme and pipeline decisions in Sections 2 and 3.

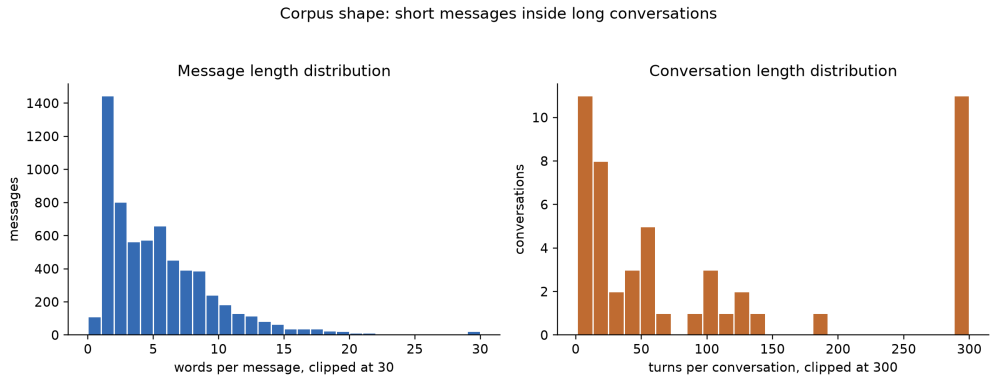
Statistic	Value
Conversations	50
Messages (non-empty)	6,483
Mean tokens / message	5.6
Median tokens / message	5
Max tokens / message	59
Close- or good-friend dyads	42 / 50
Messages with an offensive-lexicon token	1.8% (115)
Messages with Chinese script	0.1% (8)
Unattributed speaker slots	28% (1,416 / 4,983)
Conversations with message/speaker length mismatch	20 / 50

1.1 Properties

Most of the corpus is talk between close friends: 42 of 50 are close or good friends, with only a handful of colleagues, classmates, family members, or partners. Affectionate insulting and teasing ("you idiot") is the expected register between close friends and not really an edge case, and a scheme that cannot distinguish actual harm from teasing would mislabel much of the data.

The median message is only 5 tokens (mean 5.6, max 59), messages this brief are compressed, leaving little room for the surrounding linguistic cues that disambiguate tone. So the tone often has to be read from the surrounding turns and from who is talking to whom and for genuinely ambiguous cases we need the uncertainty representation mentioned in Section 2.3.

Singlish particles appear constantly: ah (104), sia (40), wah (26), lah (21), leh (18). They appear so often so it was decided that a dedicated Singlish tag (`SG`) is better than dispersing these items across other categories. Several of these are context-dependent like *ah* and *ya*, are particles in some messages and ordinary English in others. The language scheme resolves the choice between `EN` and `SG` by seeing how the token is used in the sentence.



Only 1.8% of messages (115 of 6,483) contain any token from a broad offensive-language lexicon, and inspection finds targeted hate speech to be effectively absent from the sample. This matters in two places: evaluation, and where the offensive boundary is drawn. With harm this rare, aggregate accuracy is uninformative, since a model that labels everything `neither` scores highly, and the meaningful signal lies in the minority of harm-bearing and ambiguous cases. The offensive boundary rule (the swear word test) determines most labels on this axis, since the majority of the 1.8% consists of affectionate or venting profanity that should resolve to `neither`.

Chinese-character messages are rare (∼0.1%, 8 messages), and romanised Mandarin (pinyin) is absent. A scan for distinctive pinyin items (nihao, xie xie, jiayou, . . .) finds zero hits. Other non-English content words are sparse. During eda 8 Malay and 13 romanised-Hokkien token hits and no Tamil script were found so the corpus's multilingual character lives mostly in its Singlish layer rather than in sustained code-switching. The practical implication, developed in (Section 3.7), is that the realistic Chinese-handling challenge in this data is script tagging, which both models manage. The pinyin convention in the scheme is included for completeness but cannot be meaningfully tested.

1.2 Data quality issues

The data has quality problems that limit what can be annotated.

- Speaker attribution is unreliable: 1,416 of 4,983 speaker slots (28%) are null or unattributed, and in 20 of 50 conversations the message list and speaker list differ in length, so the two cannot always be aligned turn-for-turn. Because the distinction between mock and genuine aggression often depends on who is speaking to whom, missing speaker information can leave a harm judgement undecidable. In these cases the annotation scheme assigns the `uncertain` label. Operationally, the pipeline looks up the speaker by turn index and falls back to `None` when the index exceeds the speaker list's length.
- Turns and messages are not the same unit, and some turns are not annotatable: The data contains non-message artefacts, such as reply placeholders ("In reply to this message") and self anonymised stand-ins ([NAME], [LOCATION]) that must be handled explicitly. The annotation guidelines exclude placeholder turns from harm annotation and tag anonymisation stand-ins as named entities or `OTH` as appropriate. The pipeline does not yet enforce these exclusions (Section 3.7).

1.3 Summary

This corpus is an unusual profile for a harm annotation task as it consists of short, intimate conversations with a very low harm base rate, sparse and mostly affectionate profanity, almost no hate speech, and noisy speaker attribution. The main difficulties are telling hostile wording apart from friendly intent, avoiding

overflagging profanity used to vent or bond, handling genuinely ambiguous or speaker-less messages, and evaluating sensibly when harm is this rare. The rest of the report deals with these one by one.

2 Annotation Scheme Design

This section gives the design rationale for the two annotation schemes and the reasoning behind the decisions. The operational instructions an annotator follows (decision trees, worked examples, edge-case rules) are in the annotation guidelines.

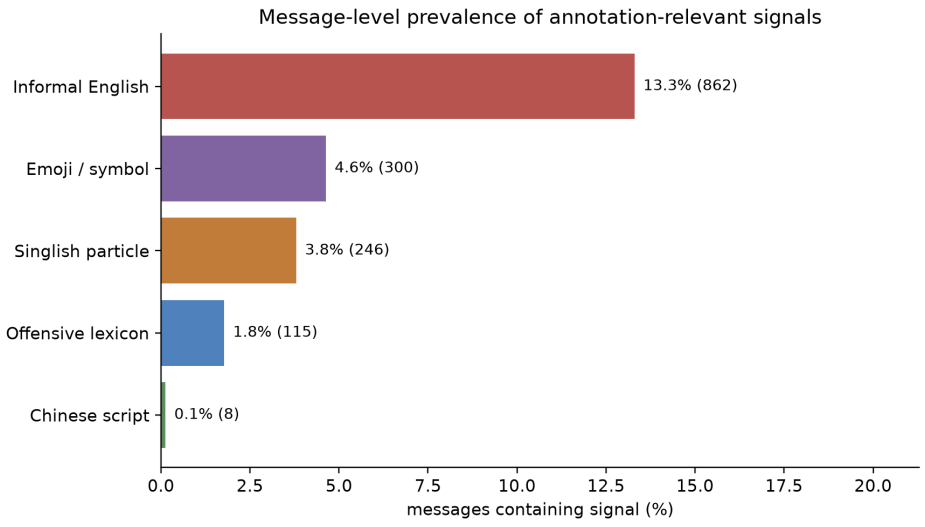
1. Label what a token *is* separately from what a message *does* Form and pragmatic function are different questions and are kept on different axes throughout.
2. Treat annotator uncertainty and disagreement as information. Where the data is ambiguous, the scheme records that ambiguity and does not force it into a false consensus (Section 2.3).

2.1 Language identification

Language identification is performed at the token level. Each token receives exactly one label, and the labels are mutually exclusive. There is no intra-token "mixed" option, because code-mixing in this data occurs across tokens within a message.

The tagset is `EN`, `ZH`, `MS`, `TA`, `HK`, `SG`, `NE`, `OTH`, `UNK`. Most tags are conventional, three decisions are specific to this setting.

SG as a functional category: Singlish colloquial items, including both discourse particles (*lah, leh, lor, sia, ah*) and lexicalised borrowings (*paiseh, kiasu, sian*), are collapsed into a single `SG` tag, assigned by function. Annotators cannot be expected to know that *paiseh* is Hokkien-derived while *lah* is Chinese-derived, and requiring etymological distinctions would introduce disagreement unrelated to the annotation goal. That goal is to mark the linguistic variety used in contemporary Singaporean conversation, and for that purpose the judgement that a token is a Singlish colloquial item is the right granularity. Since particles and borrowings function as conventional elements of Singlish regardless of origin, a single category is easier to apply consistently.



Informal English is `EN` by rule: Texting shortcuts and internet slang (*u, dat, alr, tgt, idk, lol, wtf*) are tagged `EN`. In a corpus this compressed, sending abbreviations to `UNK` would misclassify a large fraction of ordinary

English.

Romanised Mandarin is `ZH`*:* Pinyin (*ni hao, xie xie*) is tagged `ZH`, with the boundary case that a romanised item conventionalised as a Singlish borrowing (e.g. *jiayou* used as encouragement) is `SG` by function. As Section 3.7 notes, this convention is near-untestable on the present corpus because romanised Mandarin is almost absent from it but it is still included for consistency with the `ZH` definition.

Where a token could plausibly take more than one label, resolution is two-tier and not a simple priority list. A named entity is `NE` even when it is also a common word (a brand that happens to be an English noun), and emoji, URLs, numbers, punctuation, and non-lexical laughter are `OTH` even though laughter is alphabetic. These overrides are applied first. The remaining choice among `EN`, `SG`, and the other languages is a functional judgement and `SG` does not automatically take priority over `EN`. A token such as *ya* or *ah* is Singlish only when it functions as a particle in that message, and is otherwise informal English. We cannot simply encode these in precedence since that would just misclassify items but making a judgement on it keeps a label accurate.

2.2 Harm annotation: two independent axes

The harm scheme's defining choice is that it asks two independent questions about each message instead of just producing a single harm label.

Axis 1 - `harm_type` (form and content):The values are `hate_speech`, `offensive`, and `neither`. This axis follows the three-way distinction of Davidson et al. [1], who showed that conflating hate speech with offensive language is a primary source of error in harm detection, since much of what classifiers flag as hate speech is offensive but not targeted at a protected group. The scheme keeps that separation. `hate_speech` is reserved for attacks on someone because of a protected attribute (race, religion, gender, sexuality, nationality, disability), `offensive` covers profane, vulgar, or insulting language that is not so targeted, and `neither` is the majority case.

The key rule on this axis is the offensive boundary, referred to as the swear-word test. A swear word alone does not make a message offensive. Bare exclamatory profanity that vents the speaker's own feeling ("ugh, damn"; "so sian") is `neither` and removing the swear word leaves a message that merely reports annoyance. A message is `offensive` only when the profanity or insult has real force, either directed at a person or used as a deliberately derogatory evaluation ("this is fucking garbage"). This rule prevents the scheme from over-flagging the ordinary, affectionate profanity that saturates casual chat, and it is the form-side complement to the function axis below.

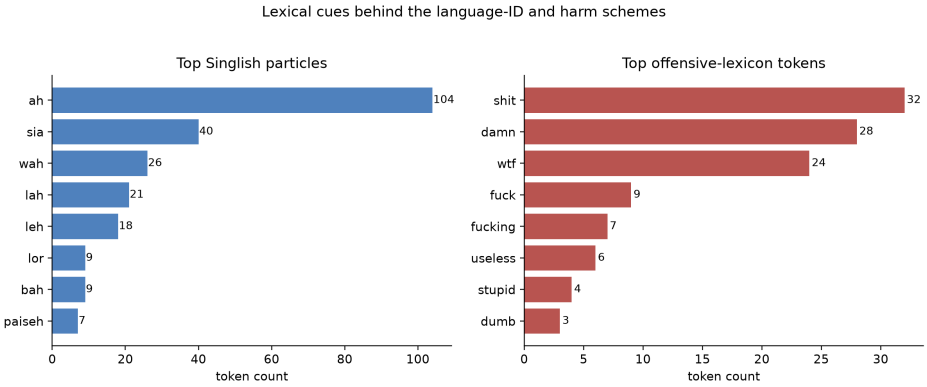
Axis 2 -`aggression_function`: The values are `genuine_aggression`, `mock_aggression`, and `none`, judged independently of Axis 1. This axis exists because the same words do different things depending on who says them to whom. `mock_aggression`, in which hostile form functions as teasing, banter, or bonding between people who are clearly close, since the corpus is dominated by close friends who insult each other affectionately. `genuine_aggression` is a real attack, threat, or expression of anger, and requires no profanity at all. A cold "don't ever message me again" is genuine aggression with a `harm_type` of `neither`. This example shows the axes are independent: any combination of form and function can occur.

A single collapsed harm label would average form and function together and lose exactly the distinctions that matter. The messages "you idiot" between close friends (`neither` with `mock_aggression`), profanity about food (`offensive` with `none`), and a calm serious threat (`neither` with `genuine_aggression`) would all end up with roughly the same label. Splitting the axes keeps each judgement clean and, as Section 3.6 shows, makes model failures interpretable, because a model's errors land on a specific axis.

The `target` field (`self`, `addressee`, `third_party`, `group`, `none`) records who the harm or aggression is aimed at, with the consistency rule that if both axes are inert (`neither` and `none`), the target is `none`. The converse rule, that a present aggression must have a non-`none` target is checked and recorded in the pipeline's validator (Section 3.5).

Harmful content in non-English terms: Because offensive force in this register often arrives through Malay, Hokkien, or romanised-Mandarin items rather than English profanity, the scheme explicitly anticipates profanity and slurs that an English-only reading would miss. The corpus bears this out at small scale: the non-English vulgarities it does contain (a bare "KNN" message; a "cb" used in banter) are invisible to the English offensive lexicon behind the corpus statistics in Section 1.

The form/function split applies unchanged across languages. A romanised Hokkien vulgarity used to vent is the same `neither` case as the English "damn", whereas the same term aimed at a person is `offensive`. Where an annotator suspects but cannot confirm that a non-English term is harmful, the guidelines specify a fluency fallback that mirrors the `UNK` logic. The annotator labels on the evidence that can be read, marks the confidence `uncertain`, and notes the token for review by a fluent speaker, which is better than passing the term as clean or guessing it into a positive label. This makes the limits of any single annotator's language coverage explicit.



2.3 Representing uncertainty

Every message carries a per-annotator `confidence` field (`certain` or `uncertain`). What we need to care about is what to do when annotators disagree. So we provide something that preserves the disagreement as a soft label rather than resolving it by majority vote. A message that two annotators read as playful and one reads as cutting is recorded as that split, not collapsed into a single confident label.

Disagreement comes in two kinds here. *Fixable disagreement* arises when a rule was unclear and annotators split because the guidelines were ambiguous. The remedy is to clarify the rule so the split does not recur. *Irreducible disagreement* arises when the message itself is genuinely ambiguous. The message "you useless" between close friends is readable as affectionate teasing or as a real jab, and no rule can settle the question, because the message itself is ambiguous and no guideline would fix that. The first kind should be reduced through guideline iteration but the second kind should be retained, because the split is real information about the message

Davani et al. [2] argue against majority vote as the default aggregation for subjective annotation, showing that modelling annotators' distinct perspectives retains information that majority voting discards. Uma et al. [3], surveying the area, make the broader case that annotator disagreement frequently reflects genuine ambiguity and that soft labels are the appropriate representation. Applied to this corpus, forcing a single

confident label onto an irreducibly ambiguous message manufactures false certainty, and a model trained on falsely confident labels will over-flag harmless messages later. Keeping the split is truer to the data, and it serves the end goal: a system that doesn't over-police ordinary conversation.

The other design choices, like the two axes, the swear-word test, and the mock-aggression category, each of these narrows down where ambiguity can hide, so whatever uncertainty is left comes from the messages themselves. This report specifies the aggregation mechanism but does not actually implement it: no multi-annotator round was run for this task, so the fixable/irreducible distinction above is argued from the corpus's characteristics. Applying it, through a small doubly-annotated pilot scored for where splits are fixable versus irreducible, is the natural first step in Section 4.

2.4 Relation to existing frameworks

Existing hate-speech schemes such as Davidson et al. [1] already distinguish hate speech from merely offensive language, a separation this scheme retains, but they do not model playful aggression. Likewise, standard language-identification tagsets offer no natural home for Singlish discourse particles, which are pervasive here and function as a coherent category regardless of etymology. So each scheme adds the axis its base framework lacks, the aggression-function axis on the harm side and the `SG` category on the language side, while inheriting the parts that are already well established.

3 Annotation Pipeline: Design and Analysis

This section describes the LLM-assisted annotation pipeline built for the two schemes, the reasoning behind its structure, and an analysis of where it succeeds and fails. The pipeline's main job is diagnostic: to find out whether a small local model can do this annotation, and if not, which parts break.

3.1 Architecture: one shared task, two backends

The two required scenarios, Scenario A (a hosted API, `gpt-4o-mini`) and Scenario B (a local open-source model, `qwen2.5:3b` via Ollama), share a single annotation core. Everything that defines the task: the two prompts, the two validators, and the control flow, resides in a single module (`annotators.py`). Each scenario contributes only a backend adapter with a minimal signature:

```
call_fn(system: str, user: str) -> str
```

The adapter returns the model's raw text. Scenario A's adapter wraps the OpenAI client and Scenario B's wraps a local Ollama HTTP call, nothing else differs between them.

Keeping the task structure identical means any difference in results comes from the models, not the setup. Per-model prompt tuning is legitimate and can be reported as such, since a prompt may reasonably be adapted to a model's quirks, but the structural decomposition of the task must remain shared for the comparison to be interpretable.

3.2 Task decomposition: two single-task calls

Each message is annotated in two separate model calls, one for language identification and one for harm, rather than in a single call that returns everything. Within `annotate_item`, the message is first tokenised (in code, not by the model), then the token list is sent for language tagging, then the raw message plus context is sent for harm annotation.

The first reason is failure isolation. Language identification only needs one tag per token whereas harm annotation in our case has 4 fields. These will have different failures show up so if we use a joint prompt, a malformed tag array can corrupt the harm object in the same response, so a single JSON error takes down both tasks. Splitting them means a mangled `langid` array cannot damage an otherwise valid `harm` judgement, and vice versa. Each block fails, and is validated, on its own.

The second reason is what the split asks of small models. A 3B model that is asked simultaneously to emit a length-exact token array and reason about pragmatic function has to juggle two unrelated output formats at once. Separating the calls lets each prompt carry only the context its task needs. Language identification is context-free (it never sees the harm tagset or the conversation) but the harm call receives the relationship and up to 3 preceding turns. This also keeps each prompt shorter, which makes it easier for small models to keep to the format.

The cost is two calls per message instead of one. For an annotation-assist tool, where correctness and interpretability matter more than latency this is a fair tradeoff

3.3 Prompt design

Both prompts are single-task and instruct the model to return strict JSON. The below design choices were taken:

- The language prompt states the token count and asks for exactly that many tags. This is an easy and low cost guard against common small-model failure, in which tags are dropped or added so that the array no longer aligns to the tokens. Strong models dont really need it but it helps the weak models
- Each prompt carries a small number of examples chosen to encode the difficult rules. The language prompt shows a particle-and-borrowing mix and the disambiguation of *ya* to `EN`, and the harm prompt shows the form/function split directly, pairing `neither` with `mock_aggression` for friendly teasing and `offensive` with `none` for profanity about food. Small models follow examples better than rules, so the hard rules are shown rather than just stated
- An explicit context-bleed warning in the harm prompt as the harm call receives surrounding turns, the model may read the playfulness or heat of the context as a property of the target message. The prompt states that laughter, emoji, capitalisation, and the tone of surrounding turns are not themselves aggression, and that a message with no aggressive tone is `none` regardless of the surrounding tone. This puts the guidelines' form/function rule directly into the prompt. This was explicitly done after observing that a message with 'HAHAHAHA' was considered to be aggression due to the capitalisation.

Output is also JSON-constrained at the transport layer, via `response_format={"type":"json_object"}` in Scenario A and Ollama's `format:"json"` in Scenario B. A permissive parser falls back to extracting the first `{...}` block, the API never needs it, but it rescues the occasional local-model reply that wraps its JSON in prose

3.4 Model selection

The comparison is `gpt-4o-mini` against `qwen2.5:3b` and it tests whether a small, locally runnable model is good enough to assist with annotation locally, at zero marginal cost and with full data control, and, if not, whether its failures can be corrected cheaply or require a larger intervention.

3.5 Validation layer

Two shared validators run on every record.

- `validate_langid` flags length mismatches (tags $\neq$ tokens) and any tag outside the fixed tagset, and only builds token–tag pairs when the lengths align.
- `validate_harm` checks all four fields against their enums and enforces two consistency rules. First, if `harm_type` is `neither` and `aggression_function` is `none`, then `target` must be `none`. Second, since an aggressive act is directed at someone, any present `aggression_function` (`genuine_aggression` or `mock_aggression`) must carry a non-`none` target. The second rule is restricted to the two valid aggressive values so that it never fires spuriously on missing or malformed output.

Every problem is recorded in an issues field on the record. This distinction matters for the analysis below, because the validators are what allow failure modes to be counted and categorised.

3.6 Results: where the models succeed and fail

The observations below come from running both models over the same N=10 sampled turns, within the 5 to 10 turn range the task specifies as sufficient to demonstrate the pipeline end-to-end. These are observations from a small set of samples to show where failure occurs. What matters is which failures show up and how they split into fixable and unfixable.

3.6.1 GPT-4o-mini

On this sample, `gpt-4o-mini`'s language tags are clean: 0 of 10 records are flagged by the validator. Its harm judgements are clean on 9 of 10 records; on the remaining one it trips the same mirror-validator inconsistency as qwen on that turn (`aggression_function` present, `target: none`),a reminder that the reference outputs are not verified gold labels. Reference outputs were read and checked against the annotation guidelines for all 10 sampled turns; no second annotator or blind check was used, so "reference" here means verified by me, not independently validated. With that caveat, gpt-4o-mini tracks the guidelines on the cases the scheme is designed around. It correctly separates offensive words from aggressive acts, tagging profanity about food or the police as `offensive` with `none` not as aggression, and it does not overread playful context as hostility. It is a usable reference for characterising the smaller model.

3.6.2 Qwen2.5:3b

Qwen's failures partition into those post-processing rules can fix and those it cannot.

Post-correctable failures (structural output errors): These are failures in the shape of the output, detected just by inspecting the output.

- *Spurious `ZH` on English tokens:* Qwen's dominant language error is tagging plain ASCII English words as `ZH` (e.g. *came*, *twice*, *chat*, *very*, *rojak*, *inside*). Across the sample this happens 18 times, spanning 6 of the 10 records, making it the most common error qwen makes on this axis. A `ZH` label on a token containing no CJK codepoints is provably wrong, so a codepoint filter that retags `ZH` when the token contains no Han characters fixes the 12 instances in length-aligned records automatically. The remaining 6 sit in the two length-mismatched records, where tags cannot be attributed to tokens until the alignment itself is repaired.
- *Out-of-tagset labels:* Qwen occasionally emits invented labels that are not in the tagset: a single `OOTH` in one record, and `PUNCT` repeated three times in another. These instances are caught by `validate_langid`.

- *Length mismatches.* The length mismatches arise where the model's tokenization diverges around punctuation and symbol characters — the $ in $25 (row 25) and sentence-final punctuation (row 18).

All three are either correctable or reliably caught, so the validators turn these problems into visible entries in the issues field instead of letting them go unnoticed

Non-correctable failures (judgement errors): These are failures in the content of the judgement and are unrecoverable from the output.

- *Profanity under-detection:* This is the most consequential failure for a harm tool. Of the 2 profanity-bearing messages in the sample that the reference flags as `offensive` (e.g. "... very stupid", "... the police better do their job"), qwen flags neither, flattening both to `neither`.No rule can repair this after the fact: the model got the judgement itself wrong, and a neither label carries no trace of what was missed. Addressing it requires a prompt-level or model-level intervention.
- *Aggression over-firing.* The converse failure occurs on the function axis. Qwen assigns `mock_aggression` to plainly benign, particle-heavy messages (e.g. "why leh dw work w me ah", "... rojak inside leh") that the reference reads as `none`. 3 of the 10 records also trip the mirror validator (aggression present with `target=none`). The likely cause is that qwen treats Singlish surface features such as *leh*, *mah*, and *ah* as hostility cues.

The two judgement failures point in opposite directions: qwen misses genuine profanity while reading ordinary Singlish banter as aggression.. The two-axis scheme makes this pattern observable, because the errors land on different axes (form and function respectively), whereas a single collapsed harm label would have merged them into noise.

Overall, the local model proved adequate for assisting with structured language annotation but substantially less reliable for pragmatic harm judgements but human review remains essential for the latter.

3.7 Prompt iteration and limitations

The pinyin rule is a completeness fix, not a measured improvement: The language guidelines treat romanised Mandarin (pinyin) as `ZH`, and the language prompt was extended to encode this explicitly, with the boundary case that a romanised item conventionalised as a Singlish borrowing (e.g. *jiayou*) is `SG` by function. This aligns the prompt with the annotation convention it implements and is verifiable by reading the prompt. The rule matches the guidelines, but the corpus gives it almost nothing to act on, so it remains unverified in practice. So the corpus cannot trigger the rule, and the realistic Chinese-tagging challenge in this data is script handling, which both models already manage.

The same status applies to `hate_speech`: the corpus's near-absence of targeted hate speech (Section 1) means the `hate_speech`/`offensive` boundary in Axis 1 is defined and prompted for but essentially unused on this sample. Like the pinyin rule, it is correct by construction but untested here.

Not every guideline rule is encoded in the prompts. The pipeline shares the guidelines' label schemes, but several operational rules exist only as annotator instructions: placeholder turns and anonymisation stand-ins are not excluded or special-cased (the tokenizer even splits [NAME] into three tokens), the per-turn speaker field is recorded in each output record but never enters the harm prompt, so the model is not told who is speaking and cannot apply the guidelines' lean-uncertain rule for speaker-less turns; and the fluency fallback for unconfirmable non-English terms (Section 2.2) and the rule that reported or quoted insults are not themselves offensive are likewise prompt-absent. These are scope cuts for a diagnostic pipeline, but each can shift labels, so the failure analysis above should be read against the prompts as they are, not the guidelines as written.

The local model does not reproduce at greedy decoding: Repeated `temperature=0` runs of `qwen2.5:3b` over identical inputs do not reproduce, language tags drift between runs on tokens the prompt never addresses. Any naïve before-and-after comparison of two local runs is therefore dominated by run-to-run variance, which is why the pinyin edit could not be A/B-tested even in principle on a small sample. Annotation systems are expected to produce stable outputs under deterministic decoding, and a model whose output is unstable at greedy decoding can't be trusted to annotate unsupervised. For the harm task, qwen's stable behaviour, the profanity under-detection that recurs identically across runs is the bigger problem since it shows up every run.

All counts in Section 3.6 come from the same N=10 sample, scored deterministically. The sampler ranks turns by particle density, profane-lexicon hits, and script presence, which is why the sample is dense in exactly those features and contains no pinyin. The figures illustrate failure modes, which are stable across runs; they are not proposed as agreement metrics. Turning them into measurements, which requires a larger sample, multiple runs per model, and a proper agreement statistic against a fixed reference.

4 Future Directions

The pipeline in Section 3 is for diagnostic purposes, and its results indicate the following next steps, starting with the cheapest.

4.1 Rule-based post-correction for output errors

The language-identification failures classed as post-correctable in Section 3.6 should be corrected in code, since they can be caught from the output itself. The most direct correction is a CJK-codepoint filter. A `ZH` tag on a token containing no Han characters is necessarily wrong and can be automatically retagged, to `EN` for ASCII-alphabetic tokens and `OTH` otherwise, or routed to a re-ask. On the observed sample this alone removes the small model's dominant language error. The invalid-tag and length-mismatch checks already report their problems as `issues`; the natural extension is a bounded re-ask loop, in which a validator failure triggers a single re-prompt stating the specific error. Code should handle the mechanical errors so prompt work can focus on the judgement ones.

4.2 Closing the profanity-detection gap

The profanity under-detection in Section 3.6 is a judgement error and cannot be filtered away, so it is the right target for prompt-level and model-level effort. Start with a revised harm prompt carrying more offensive-boundary examples in the code-mixed register, tested before-and-after against a fixed reference. If that doesn't close the gap, try a 7B–8B model from the same family to see whether the problem is capacity or prompting.

Because the task decomposition is shared (Section 3), a prompt revision here is an acceptable per-model adjustment, as long as it's documented and the structural comparison remains clean. The aggression over-firing should be probed in the same way, since it probably has the same root cause: the model reads Singlish particles as hostility cues

4.3 Reproducibility and measurement

Section 3 reports failure modes because the sample is small and the local model does not reproduce at greedy decoding. Turning the observations into measurements requires a larger annotated sample, multiple runs per model to quantify run-to-run variance directly (itself a reportable reliability property of the local model), and a proper agreement statistic against a fixed reference. Krippendorff's α is the natural choice, as it accommodates multiple annotators, tolerates missing labels, and is compatible with the soft-label

representation of Section 2.3 instead of just collapsing to a single gold label. Reporting α separately per task (language, harm type, aggression) keeps the axis-specific failure structure visible.

4.4 Evaluation under class imbalance

With harm present in under 2% of messages, any aggregate metric is dominated by the `neither` majority and hides the errors we care about. Future evaluation should therefore be disaggregated into per-class precision and recall on `offensive` and `hate_speech` and, most importantly for a harm tool, the false-positive rate on benign messages, reported separately for the affectionate and banter cases that dominate the corpus. The contested cases should additionally be evaluated as contested, consistent with the soft-label design.

4.5 Non-English harm coverage

The scheme anticipates non-English profanity and slurs (Section 2.2) and provides a fluency fallback, but the pipeline does not yet give the model explicit support for them. A modest addition is a curated lexicon of the common non-English offensive terms in this setting (romanised Hokkien and Malay in particular), supplied to the harm prompt as reference material and paired with human adjudication for the flagged-but-unconfirmed cases the fallback routes out. This targets the kind of false negatives that hurts a harm system most, and one an English-centric model is especially likely to make.

5 Optional extension: disclosure annotation

5.1 Definition

Disclosure here means a speaker voluntarily revealing personal information like facts, feelings, or thoughts that the listener could not otherwise know. This definition follows the social penetration theory [4], where they explain disclosure as something that varies in breadth and depth as relationships develop, and the coding scheme follows the ideas by Barak and Gluck-Ofri [5], which separates disclosure of information, thoughts and feelings. The scheme keeps the same design principles as the harm scheme: independent dimensions, message-level units, and the uncertainty representation of Section 2.3 carried over.

So for each message we take into account these following dimensions:

- `disclosure_type`: `informational` (facts or events from one's life), `emotional` (feelings or inner states so bare mood venting like "so sian" counts, since a feeling is disclosed even in three tokens), `cognitive` (personal opinions and beliefs), or `none` (questions, logistics, reactions). If mutiple categories are applicable then the deepest is labelled.
- `depth`: `low` (routine daily life), `medium` (personal but not sensitive), `high` (health, family or relationship conflict, fears, secrets, money, failure). Depth judges what is revealed, not how strongly it is worded: "lol my parents might divorce" is high.
- `subject`: `self` or `other`—passing on a third party's private information ("she told me she's failing, dont tell anyone") is a different act from disclosing one's own, and the one with privacy implications.

A fourth dimension, elicitation (spontaneous vs. prompted), would be needed to study reciprocity. For now it is left out of the implementation since reciprocity is better derived at conversation level than annotated per message.

Guidelines for this layer would reuse the structure of the main guidelines document. The ambiguous cases to cover are venting vs. emotional disclosure (resolved above: venting counts, at low depth), hyperbole ("i

want to die lol"), disclosure carried by the non-English part of a code-mixed message, reported disclosure of others, and anonymisation placeholders capping what can be identified.

5.2 How this corpus works well

The dataset attaches conversation-level self-report about disclosure behaviour (`share_info_self`, `share_emotions_self`, `trust_with_secrets`, `comfort_sharing_thoughts`). Message-level disclosure annotation would give these self-reports something to be checked against: whether people who report high emotional sharing actually produce more emotional and high-depth messages, and whether `trust_with_secrets` predicts the rate of `subject=other` disclosures. That comparison between what people say they share and what their messages show is a research question this corpus is well placed to answer.

5.3 Pipeline extension and a small run

Because the pipeline decomposes into single-task calls (Section 3.2), disclosure slots in as a third call: one prompt, one validator (enum checks plus the consistency rule that `type=none` forces `depth` and `subject` to `none`, and a present disclosure requires both), enabled by a `-disclosure` flag in both scenarios so the core task is not affected.

Both models were run over an identical set of 10 sampled turns – the sampler's top-ranked turns 1–10, a different set from the harm analysis in Section 3.6, which used turns 11–20 of the same ranking – and the result repeats the pattern of Section 3.6 on a new task. The structural layer was clean for both (0 of 20 records flagged): the disclosure schema has no length-alignment burden, which is where the language task's structural failures came from. The models agree on 6 of 10 turns, and every disagreement is a judgement difference from the smaller model:

- `qwen` labels "damn useless" (an evaluation) and "im so close to 5000 points" (the speaker's own state) as `none`, where `gpt-4o-mini` reads them as `cognitive` and `emotional` disclosure. This is the disclosure analogue of the profanity under-detection in Section3.6.2: the small model misses content that is present.
- "he was damn pissed" reveals a third party's state; `gpt-4o-mini` labels `subject=other`, `qwen` labels `subject=self`. `qwen` also reads a message that mostly asks a question ("you reach home already?") as an emotional disclosure with `subject=other` mislabelling a question directed at the addressee as third-party disclosure, which the scheme rules out

`gpt-4o-mini`'s labels track the scheme on all 10 turns, including the subject rule. So the Section 3.6 conclusion carries over: the local model is structurally reliable but not yet trustworthy on judgement, and the judgement gap is in the same direction (under-detection) across both tasks. One caveat: the sampler ranks turns by harm-related features (Section 3.7), so this sample is not representative for disclosure, a proper run would sample for disclosure-bearing turns instead.

References

[1] Davidson, T., Warmsley, D., Macy, M., & Weber, I. (2017). Automated Hate Speech Detection and the Problem of Offensive Language. *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*.

[2] Davani, A. M., Díaz, M., & Prabhakaran, V. (2022). Dealing with Disagreements: Looking Beyond the Majority Vote in Subjective Annotations. *Transactions of the Association for Computational Linguistics*.

[3] Uma, A. N., Fornaciari, T., Hovy, D., Paun, S., Plank, B., & Poesio, M. (2021). Learning from Disagreement: A Survey. *Journal of Artificial Intelligence Research*.

[4] Altman, I. and Taylor, D.A., 1973. Social penetration: The development of interpersonal relationships. Holt, Rinehart & Winston.

[5] Barak, A. and Gluck-Ofri, O., 2007. Degree and reciprocity of self-disclosure in online forums. CyberPsychology & Behavior, 10(3), pp.407-417.